

Treasure Island Hurricane Damage Detection

Progress report — DINOv2 pipeline fix, combined-signal detection, and per-building damage scoring

Prepared September 2026

Repository: github.com/sai0215/ImageClassTreasureIsland

Summary

The existing pipeline compared pre/post-hurricane imagery of Treasure Island using SSIM and a single whole-image DINOv2 embedding. This round of work addressed a specific concern: whether a small, localized change (down to single-pixel-scale damage) could be lost in a foundation-model comparison. Three things came out of that investigation: (1) a concrete bug in the DINOv2 pipeline that was discarding up to half of each image before analysis, now fixed; (2) a combined SSIM + DINOv2 signal that cross-checks fine-grained and semantic evidence of change; and (3) a per-building analysis pipeline, adapted from Microsoft's HASTE disaster-response platform, that scores individual buildings rather than whole scenes — directly addressing the original small-feature-sensitivity concern.

1. Bug found: DINOv2 was only seeing part of each image

The Hugging Face image processor for facebook/dinov2-base defaults to resizing the shorter side to 256px and then center-cropping to a fixed 224×224 square. This was happening silently on every comparison. Measured against the three actual scene pairs:

Pair	Source size (w×h)	Width kept	Height kept	Area kept
Maxar pre vs post	1402 × 1600	87.5%	76.7%	67.1%
NOAA Helene vs Milton	1600 × 1600	87.5%	87.5%	76.6%
Sentinel-2 pre vs post	1264 × 2000	87.5%	55.3%	48.4%

For the Sentinel-2 pair specifically, less than half the image (48.4%) was ever reaching the model. On top of that, the surviving center region was downsampled by roughly 5× before patch tokenization, so each 14×14 DINOv2 patch actually represented ~70–100 original pixels per side — and the resulting 16×16 similarity grid was then bicubic-upsampled back to full image size, smearing whatever weak signal survived over a large area.

2. Fix: full-frame, overlapping-tile dense embedding

dinov2_compare.py was rewritten with two changes: (a) the whole-image CLS embedding now runs with center-crop disabled, so the global similarity score reflects the entire frame; (b) the fine-grained dissimilarity map is now built by tiling each image into overlapping 224×224 windows (50% overlap), embedding each tile at native resolution, and stitching the per-tile results into one full-resolution map with overlap-averaging — replacing the single-pass, heavily-blurred 16×16 grid.

Pair	Tiles used	Tile size	Global CLS cos-sim	Mean tiled dissimilarity
Maxar pre vs post	168	224px	0.723	0.410
NOAA Helene vs Milton	196	224px	0.918	0.351

Sentinel-2 pre vs post	187	224px	0.892	0.405
------------------------	-----	-------	-------	-------

Runtime: all three pairs process in ~40 seconds total on Apple Silicon (MPS).

Maxar: 2023-11-04 (pre) vs 2024-10-10 (post-Milton) — DINOv2 embedding similarity

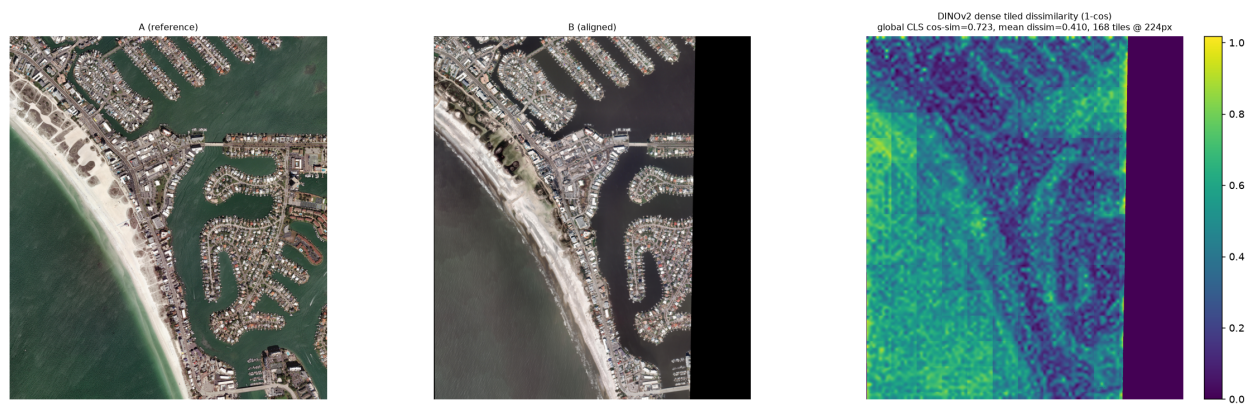


Figure 1. Maxar pre/post comparison after the fix — the dissimilarity map (right) now covers the full frame, including the masked-out alignment border (solid purple strip), with spatial texture tracking real structure instead of a coarse blur.

3. Combined SSIM + DINOv2 signal

combined_compare.py pairs the two signals as complements: SSIM (native resolution) is sensitive to small localized changes but also fires on noise, sensor artifacts, and residual misalignment; the new dense DINOv2 map is coarser but more semantically robust. Each map is Otsu-thresholded independently, and pixels are bucketed into confirmed (both agree), fine-only (likely noise), and semantic-only (broad shifts such as water color, without a matching fine-pixel spike).

Pair	SSIM flagged	DINOv2 flagged	Confirmed	Fine-only (likely noise)	Semantic-only (broad)
Maxar pre vs post	54.0%	49.5%	32.2%	21.8%	17.3%
NOAA Helene vs Milton	42.4%	40.4%	25.6%	16.8%	14.7%
Sentinel-2 pre vs post	28.0%	51.3%	24.9%	3.1%	26.4%

Note: Otsu thresholding splits each map at its own bimodal point, which for a busy real scene can sit low — so "confirmed" here reads as broad district-level shift rather than tight, isolated damage. A percentile-based threshold (e.g. top 10–15%) is the identified next step to tighten this into a more actionable damage-candidate map.

4. Per-building damage scoring (adapted from HASTE)

To directly address small-feature sensitivity, the analysis unit was moved from whole scenes to individual buildings, following the method used by Microsoft's open-source HASTE disaster-response platform (github.com/microsoft/haste): crop imagery to one building's bounding box, mask the building polygon into the DINOv2 patch-token grid, and pool only the patches inside it into one feature vector per building. This confines each embedding to one real structure, so a small real change dominates its own analysis instead of being averaged across an arbitrary tile.

4,382 building footprints for Treasure Island were pulled from OpenStreetMap via the Overpass API and saved to footprints/treasure_island_buildings.geojson. Since no damage labels exist yet for these buildings, footprint_compare.py scores each one by the cosine distance between its pre- and post-event pooled embedding — an unsupervised version of HASTE's approach.

Validation: before trusting the pipeline against real data, it was run end-to-end against a synthetic raster built from real Treasure Island footprint geometries, with a known subset of buildings deliberately altered. Result: the ~14 altered buildings scored 0.13–0.76, while the remaining unchanged buildings scored effectively 0.0 — a clean separation confirming the crop / mask / embed / score pipeline works correctly.

Current blocker: this requires georeferenced imagery (GeoTIFF/COG with a real coordinate transform) so footprint polygons can be placed onto the raster in pixel space. The existing images/ folder only contains flat preview PNGs with no geotransform. Georeferenced sources are being obtained next.

5. Supervised damage classifier (xBD-trained probe)

train_xbd_probe.py extracts the same footprint-pooled DINOv2 embedding for every labeled building in the xBD dataset (~850,736 labeled buildings across 19 disaster events, including past hurricanes) and trains a logistic-regression probe mapping the post-event embedding to a damage class (no-damage / minor-damage / major-damage / destroyed) — the supervised counterpart to the unsupervised cosine-distance score above,

matching HASTE's own post-only-embedding-plus-linear-probe approach.

xBD requires registration at xview2.org and is not automatable, so this is a verified scaffold rather than a completed run: the full JSON-label parsing, per-building crop/mask/embed pipeline, and train/evaluate loop were tested end-to-end against a synthetic 4-class, 32-building xBD-format dataset (perfect classification on the held-out synthetic split), confirming correctness ahead of the real ~10GB dataset download.

`footprint_compare.py` already looks for a trained probe (`models/xbd_dinov2_probe.joblib`) and, once present, adds a predicted `damage_class` column alongside the existing `change_score` for every building.

6. Next steps

- Obtain georeferenced (GeoTIFF/COG) versions of the Maxar / NOAA / Sentinel-2 scenes and run `footprint_compare.py` end-to-end on real Treasure Island buildings.
- Register for and download the xBD dataset, then run `train_xbd_probe.py` to produce a real damage-class probe.
- Tighten the combined SSIM + DINOv2 signal from Otsu to a percentile-based threshold for a cleaner, more actionable change map.

All changes are committed to the project git history (4 commits this round: DINOv2 center-crop fix, combined-signal script, per-building HASTE-style scoring, xBD probe scaffold). Full source: github.com/sai0215/ImageClassTreasureIsland.